

Emergency Dispatch Priority and Coordination System

Final Project Report

ENSE707 Software Quality Assurance

Leo Cao (23211788), Fateh Bhular (23217987), Shawn Lee (23204035)

Auckland University of Technology

Table of Content

1. Project Completion and Final Implementation.....	4
1.1 Project Overview.....	4
1.2 Final Scope and Changes Since Assessment 1.....	4
1.3 Final System Architecture and Design.....	4
1.4 Final Implemented Functionality.....	4
1.5 Final Prototype Evidence.....	4
1.6 Remaining Implementation Limitations.....	4
2. Requirements Verification and Traceability.....	5
2.1 Finalised Requirements.....	5
2.2 Requirements Verification.....	6
2.3 Final Requirements Traceability Matrix.....	6
2.4 Partially Implemented, Unverified and Out-of-Scope Requirements.....	6
3. Test Execution and Final Test Results.....	7
3.1 Final Testing Scope and Approach.....	7
3.2 Final Test Cases.....	7
3.3 Test Execution Results.....	7
3.4 Failed Tests and Corrective Actions.....	7
3.5 Regression Testing.....	7
3.6 Remaining Testing Risks.....	7
4. Test Automation and Quality Engineering.....	8
4.1 Automation Selection.....	8
4.2 Automated Test Implementation.....	8
4.3 Automated Test Results.....	8
4.4 Automation Maintainability.....	8
4.5 Manual and Exploratory Testing.....	8
5. Continuous Integration and DevOps QA.....	9
5.1 Development and Integration Workflow.....	9
5.2 Continuous Integration Pipeline.....	9
5.3 Quality Gates.....	9
5.4 Smoke and Sanity Testing.....	9
5.5 CI Evidence and Limitations.....	9
6. Defect Management and Root Cause Analysis.....	10
6.1 Defect Management Approach.....	10
6.2 Severity, Priority and Status Definitions.....	10

6.3 Defect Lifecycle.....	10
6.4 Final Defect Register.....	10
6.5 Root Cause Analysis: Defect One.....	10
6.6 Root Cause Analysis: Defect Two.....	10
6.7 Root Cause Analysis: Defect Three.....	10
6.8 Corrective and Preventive Actions.....	10
7. Quality Testing.....	11
7.1 Risk-Based Selection of Quality Areas.....	11
7.2 Quality Area One: [e.g. Security].....	11
7.3 Quality Area Two: [e.g. Performance].....	11
7.4 Results and Limitations.....	11
7.5 Recommended Improvements.....	11
8. Quality Metrics and Final Release Decision.....	12
8.1 Final Quality Metrics.....	12
8.2 Requirements and Acceptance-Criteria Evaluation.....	12
8.3 Outstanding Defects and Residual Risks.....	12
8.4 Final Release Decision.....	12
9. AI-Assisted Development and Critical Reflection.....	13
9.1 AI-Assisted Activities.....	13
9.2 Representative GitHub Copilot Evidence.....	13
9.3 Human Review and Validation.....	13
9.4 AI-Related Quality Risks.....	13
9.5 Critical Reflection.....	13
10. Quality Improvement and Future Recommendations.....	14
10.1 Key Quality Lessons.....	14
10.2 Improvements to the Development and QA Process.....	14
10.3 Future Testing and Automation Improvements.....	14
10.4 Future Continuous Quality Engineering.....	14
11. Team Contribution and Professional Reflection.....	15

1. Project Completion and Final Implementation

- ☐ *Complete the software solution or quality improvement prototype proposed in Assessment 1*
- ☐ *Implement the key functional requirements and acceptance criteria*
- ☐ *Document and justify any changes to scope, requirements, architecture, technology, or implementation*
- ☐ *Demonstrate the final system through a working prototype*
- ☐ *Provide a GUI where user interaction is required, or justify an alternative interface (API, CLI, backend service, automated testing framework, or QA dashboard)*

1.1 Project Overview

There is a high volume of calls received by emergency dispatchers every day in New Zealand alone. Sometimes there are too many cases that office workers have to go out into the field to keep up with all of them (*1News Reporters, 2026*). This causes stress on the current emergency system, which underlines the need for a new response system that can effectively manage all emergency cases and coordinate response units for these cases. The Emergency Dispatch Priority and Coordination System was developed to improve the management of emergency calls, case handling, prioritisation, department routing, and response-unit coordination. The prototype from phase I allows the dispatcher to create a case and put it through the system, which assigns a priority. The case is then routed to the assigned departments, where they can view the case details on their dashboards with very quick updates. After a unit has finished a case, they can update it on their dashboard, and once all units have signed off, the case closes.



1.2 Final Scope and Changes Since Assessment 1

The original scope from assignment 1 was to develop an emergency dispatch priority and coordination system prototype that managed emergency cases, case prioritisation, case routing to departments, and response-unit coordination. The scope for the final assignment includes some changes to the features due to new findings, corrections required from the previous scope, as well as new ideas.

Scope Area	Assessment 1 Scope	Final Assignment Scope	Justification
Case recording	Dispatchers can record caller details, incident type, description, location, severity, and required departments in a form. Submitting this form creates a new case.	No scope change. The required inputs remain validated, and the case number and timestamp are generated automatically during case creation.	Case recording provides the information that is used in prioritisation, routing, unit assignment, and testing. The automatic timestamp and case number improve consistency.
Prioritisation	The system automatically determines the case priority using the department-related rules. The dispatcher's selected severity is used when there are no keyword matches.	Scope changed. An authorised dispatcher can override the calculated priority by giving a mandatory reason, which is stored in the audit log.	Priority override allows a miscalculated priority to be corrected. The original priority, updated priority, user, reason, and the timestamp should all be recorded for accountability,
Routing	Cases are routed to every department selected by the dispatcher. Multi-department cases appear in all of the selected departments' interfaces.	No scope change. Selected departments can view the case on their dashboards. Cases are excluded from unrelated departments' dashboards.	This workflow satisfies the multi-department routing scope from assignment 1. There is no external emergency-service integration in this prototype.
Unit assignment and coordination	The first available unit from each required department will be assigned to the case. If no compatible units are available, the case will be in a queue.	Scope changed. Unit assignment assigns the nearest available unit using its stored location data. If no unit is available, the case remains in the queue.	Nearest-unit selection can reduce travel distance for the selected response unit and the expected time of arrival. This is supported by location data that

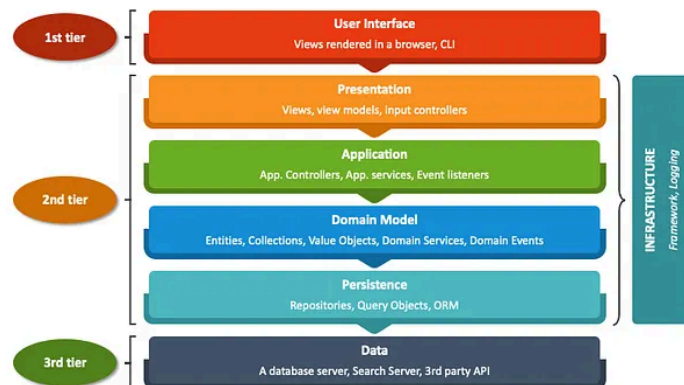
			is measurable, defined rules, and repeatable tests.
Case lifecycle, sign-off, and history	Cases use the following states: Open, In Progress, Closed. Each unit signs off individually, and the case only closes when all units have signed off. Dispatchers can search previous cases by caller name, case number, or date.	No scope change. The case lifecycle, unit sign-off, and case history are kept the same. Case closes after all required department response units have signed off. History remains searchable by the inputs mentioned.	The lifecycle prevents cases from being closed too early on multi-department cases, and keeps all states consistent throughout the case lifecycle. The history supports traceability and access to previous cases.
Role-based GUI	There are separate interfaces and restrictions provided for dispatchers, departments, response units, and the prototype admin role. Department and response unit users are restricted to their assigned scope.	No scope change. The role-based interfaces and server-side access controls are kept. The admin is prototype only and is used for testing and demonstration.	Role-specific interfaces reduce the complexity and prevent users from accessing pages they shouldn't be allowed to access. Admin is kept for the prototype for easier testing and demonstration.
Architecture & Technology	This system uses C#, .NET, Razor Pages, HTML/CSS, MSTest, and GitHub with a modular design.	No scope change. The structure, repository abstractions, priority strategy, dependency injection, and central dispatch service are kept.	Keeping the existing technology stack avoids migration and regression risks. Explaining the architecture shows how the prototype supports maintainability, extensibility, and testing.
Security and Persistence	The system uses in-memory repositories and fake accounts in text files. Operational data was lost after restarting the program. Role-based authorisation was	Scope changed. Includes permanent database storage, password hashing, encryption at rest, and a persistent audit log. The audit log records important actions related to a case,	Persistent storage prevents important data from being lost after the program restarts. Password hashing and encryption reduce the risk of sensitive data being exposed if

	implemented, but not permanent storage or encryption.	timestamps, and the responsible user.	the database is accessed directly. Audit logging improves accountability, traceability, and security review.
--	---	---------------------------------------	--

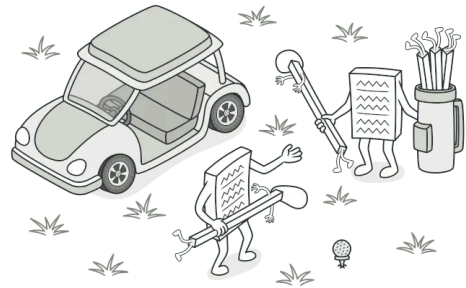
1.3 Final System Architecture and Design

There are six layers used in our system architecture: Domain, Application, Logic, Infrastructure, Web, and Authentication. This separation in the system means that each part can focus on one responsibility, while still contributing a part to the complete dispatch workflow.

The **domain layer** contains the system's main objects, with each of them controlling its own rules, functionality, and state changes in the system. For example, assigning cases, releasing a unit, or closing a case. The main objects are cases, units, departments, priorities, and case states. The **application layer** is the coordinator of the system and tells parts of the system "what needs to happen". DispatchService is an example in the application layer, where it coordinates the case creation, prioritisation, routing, and case-related tasks. The **logic layer** contains the rules about the system objects that are independent of the interface and database. For example, KeywordSeverityPriority.cs implements the priority logic in the system, where its rules can be changed independently. The **infrastructure layer** contains all of the department, case, notification and other repositories. These have direct access to the database and its chosen operations. The **web layer** uses ASP.NET Core Razor Pages to display the interface and pages of the system. This includes the sign-in page, dispatcher dashboard, department dashboard, and all other system pages. Lastly, the **authentication layer** holds files that are responsible for credential checking and making sure users can only access information that is permitted by their role.



Our system implements **two main design patterns: the repository pattern and the strategy pattern**. The repository design pattern gives an abstraction layer between the business logic and data storage. This gives a consistent way of managing data while hiding the implementation details (*GeeksforGeeks, 2026*). `ICaseRepository`, `IDepartmentRepository` and `IDispatchNotifier` all use the repository design pattern to separate the workflow from the storage. The strategy design pattern is a behavioural design pattern where a group of similar algorithms are used in a way that makes them interchangeable. This allows the algorithm to vary depending on what the client requires for their task, and enables these changes during runtime without modifying the existing code (*GeeksforGeeks, 2026*). In our system, the strategy pattern was used for case priority calculation. The `IPriorityStrategy` interface defines the framework to calculate priority, and `KeywordSeverityPriority` provides the implementation of that framework.



We used **dependency injection** to connect the different layers and components of our system architecture. All of the repositories, priority strategy, notification service, and the dispatch service are loaded when the application starts. Rather than every class creating its own dependencies created unnecessary load on the system, the components are provided to those classes instead. This reduces the coupling between components while also making testing easier because the repositories and services can be provided for testing instead of the real implementations.

The `DispatchService` was implemented as the **central service for the main system workflow**. It coordinates the case creation, priority calculation, storage, unit assignment, notifications, unit sign-off, and the reassignment of released units to the cases in queue. The intention of this service was to prevent similar workflows from being implemented in different locations of the system. The pages receive user input and display information, while `DispatchService` coordinates what should happen to the case and units (backend of the application).

Domain-controlled state was used to reduce the possibility of invalid states during runtime. The states of cases and response units are controlled by the domain objects instead of the user interface. The `Case` object controls whether the unit can be assigned, when a unit signs off, and whether the case

should be Open, Closed, or In Progress. The Unit object controls its availability and the current case assignment. With domain objects controlling their own states, it prevents invalid states. For example, an unavailable unit still being assigned to another case, or a multi-department case closing when not all units have signed off.

All of the design decisions and patterns implemented improve the quality of the system and make it easier to test and maintain. The **layered architecture helps with testing**, due to all the business logic being separated from the web interface. This allows the business logic to be tested without needing to load the interface, allowing for easier automated testing. The **repository design pattern reduces the coupling** between components because the application layer doesn't worry about how the data is stored. Overall, the design decisions make the system more modular and easier to modify without needing to also modify other unrelated layers. The persistent database, password hashing, and audit logging added improve the reliability, security and traceability of the system. This is because it prevents data loss after the program restarts, protected information is secure, and important actions on cases are recorded for accountability.

1.4 Final Implemented Functionality

These are the core functionalities that we have implemented during the first development of the initial prototype :

Requirements	Implemented Functionality	Acceptance criteria met	Evidence
FR-01 & FR-02	Emergency case recording and creation	The required caller, incident, location, severity, and department information is validated correctly. If valid, the submission creates a case with a unique timestamp and case number.	TC-01,TC-02, TC03
FR-03	Department unit management	Department users can only view their units and update unit location and their worker count. The unit availability is controlled by assignment and sign-off.	Implemented, but the final automated and browser evidence is required

FR-04	Automatic case prioritisation with authorised override	The system calculates priority using department rules. An authorised dispatcher can override the priority result with a mandatory reason and, after entering the reason, the original priority, updated priority, user, and timestamp.	TC-03, TC-04. Override implementation and tests pending
FR-05	Department routing	The cases appear on every selected department's dashboard and are excluded from unrelated department dashboards.	TC-03, TC-05, TC-09
FR-06	Nearest available unit assignment and waiting queue	The nearest available response unit is selected using its stored location data. If no unit is available, the case remains waiting until a compatible unit is free.	Existing first-available tests: TC-03, TC-06, TC-07, TC-08. Distance tests pending.
FR-07	Automatic dashboard updates	Each department dashboard only displays its relevant cases and includes new routed cases automatically. The page refreshes every 5 seconds.	TC-09. Manual browser testing pending.
FR-08	Case status management	The case follows the defined state rules (Open, In Progress, and Closed) in waiting, assignment, partial sign-off, and completion.	TC-03, TC-06, TC-07, TC-10
FR-09	Unit-availability management	Units become unavailable when assigned and available again after signing off. Two requests at the same time cannot give one unit two active assignments.	TC-03, TC-06, TC-07, TC-08, TC-10
FR-10	Assignment notifications	Each unit that is assigned receives a notification containing the case number, incident type, and location. If the notification fails, it should not undo successful case creation or assignment.	TC-03; notification-failure architecture test.
FR-11	Response sign-off and closure	Units sign off individually; if units are still assigned to a multi-department case, its state should be In Progress. The case closes after all assigned cases have signed off.	TC-10
FR-12	Case-history search	Dispatchers can search by caller name, case number, or date. Cases that match any of the fields are returned.	TC-11

FR-13	Persistent audit logging	Important actions include case creation, priority override, routing, assignment, sign-off, and closure. These are stored with the case action, user or system actor, and timestamp. Audit records should remain accessible after the program restarts.	Tests pending
NFR-SO2 & NFR-U01	Role-based access and interfaces	Dispatchers, departments, and response units can access only their allocated pages and scope. Any unauthorised page requests should be denied.	TC-12; final browser evidence required
NFR-RO3	Permanent database persistence	Cases, units, assignments, notifications, accounts, and audit records should remain accessible after the application has restarted. Stored data should be retrieved without missing or changed fields.	Restart-persistence and data-integrity tests pending
NFR-SO1	Encryption at rest	Sensitive account data should not be stored as readable plaintext in the local database. Authorised operations can encrypt, store, retrieve, and decrypt the data correctly.	Encryption and database-inspection tests pending.
NFR-SO3	Secure password storage	Account passwords should be stored using password hashing. Correct passwords should be accepted, while incorrect passwords and direct hash comparison attempts are rejected.	Authentication and password-hashing tests pending.

1.5 Final Prototype Evidence

1.6 Remaining Implementation Limitations

2. Requirements Verification and Traceability

- ☐ Review and finalise the functional and non-functional requirements from Assessment 1
- ☐ Update the Requirements Traceability Matrix
- ☐ Map requirements to implemented features, test cases, test results, and defects where applicable
- ☐ Identify any requirements that are partially implemented, unverified, or out of scope, and justify them

2.1 Finalised Requirements

From reviewing the 24 finalised requirements in the mid-term report, changes to some of the requirements would need to be made to increase the consistency, testability, clarity, and completeness. These are the requirements that need to be altered to consider the feedback provided in the mid-term report and further evaluation of each requirement:

ID	Original	Finalised	Reason
NFR-SO1	Any data that passes between users and the system must be encrypted to prevent unauthorised access.	The data during transmission between the user and the system shall be encrypted using TLS 1.2 or higher, and the data stored (account data, incident details, and caller data) shall be encrypted using AES-256.	There's no specific testable way to check if the system is encrypted, what counts as preventing unauthorised access, and to what standard. It does not specify the data during transit or data at rest.
NFR-RO1	Data from a case shall not be lost when it is being sent from the dispatcher to the selected emergency department.	The case data shall remain complete and unaltered when the dispatcher transfers data to each selected emergency department.	NFR-RO1 and NFR-RO3 are too similar; NFR-RO1 is about data moving from the dispatcher to the emergency department, and NFR-RO3 is about data being saved. The requirements need to be rephrased in order to provide a more distinct difference.

NFR-RO3	All case log entries and incident reports submitted through the system shall be saved without losing any of the data entered.	All case log entries and incident reports submitted shall be stored persistently, and when retrieving the data, the data to retrieve shall match what was submitted, with no fields missing or changed.	NFR-RO1 and NFR-RO3 are too similar; NFR-RO1 is about data moving from the dispatcher to the emergency department, and NFR-RO3 is about data being saved. The requirements need to be rephrased in order to provide a more distinct difference.
NFR-MO2	The system shall be designed in a way that allows new user roles and departments to be added without requiring modification of existing classes.	The system shall allow new user roles and departments to be created without changing the existing case creations, routing, assignment of cases, or status updates.	This requirement is too strict because modification of existing classes is improbable and could not be confirmed objectively. The new requirement restricts the changes to the core components while removing the restriction for the rest of the components.

The rest of the non-functional requirements and functional requirements were unchanged due to the ambiguity found in the previous improvement iteration of the requirements. All the finalized requirements are in **Appendix 5**.

2.2 Final Requirements Traceability Matrix

2.3 Partially Implemented, Unverified and Out-of-Scope Requirements

3. Test Execution and Final Test Results

- ☐ *Execute the planned tests against the completed system*
- ☐ *Include relevant functional and non-functional tests (unit, integration, system, acceptance, regression, usability, security, performance, or accessibility)*
- ☐ *Include positive, negative, boundary, invalid-input, and error-handling scenarios where applicable*
- ☐ *Provide at least 15 final test cases with execution results*
- ☐ *Summarise the number of planned, executed, passed, failed, blocked, and not-run tests*
- ☐ *Explain failed tests, corrective actions, regression testing, and remaining quality risks*

3.1 Final Testing Scope and Approach

3.2 Final Test Cases

3.3 Test Execution Results

3.4 Failed Tests and Corrective Actions

3.5 Regression Testing

3.6 Remaining Testing Risks

4. Test Automation and Quality Engineering

- ☐ *Identify suitable test cases for automation and justify the selection*
- ☐ *Implement automated tests using an appropriate tool or framework where suitable*
- ☐ *Report automated test execution results using logs, screenshots, reports, or repository evidence*
- ☐ *Discuss the benefits, limitations, and maintainability of the automated test suite*
- ☐ *Where automation is not appropriate, justify the use of manual or exploratory testing*

4.1 Automation Selection

4.2 Automated Test Implementation

4.3 Automated Test Results

4.4 Automation Maintainability

4.5 Manual and Exploratory Testing

5. Continuous Integration and DevOps QA

- ☐ *Describe the development and integration workflow*
- ☐ *Implement a simple CI workflow or equivalent automated quality check where appropriate*
- ☐ *Define quality gates (successful build, passing tests, linting, code review, or coverage threshold)*
- ☐ *Demonstrate smoke or sanity testing where relevant*
- ☐ *If CI/CD is not technically appropriate, justify an alternative quality workflow*

5.1 Development and Integration Workflow

5.2 Continuous Integration Pipeline

5.3 Quality Gates

5.4 Smoke and Sanity Testing

5.5 CI Evidence and Limitations

6. Defect Management and Root Cause Analysis

- ☐ *Maintain a final defect register covering defects found during development and testing*
- ☐ *Define severity, priority, and defect status categories*
- ☐ *Describe the defect lifecycle used in the project*
- ☐ *Analyse at least 3 significant defects using root cause analysis*
- ☐ *Propose corrective actions and defect-prevention measures*
- ☐ *Explain how defect analysis improved product or process quality*

6.1 Defect Management Approach

6.2 Severity, Priority and Status Definitions

6.3 Defect Lifecycle

6.4 Final Defect Register

6.5 Root Cause Analysis: Defect One

6.6 Root Cause Analysis: Defect Two

6.7 Root Cause Analysis: Defect Three

6.8 Corrective and Preventive Actions

7. Quality Testing

- ☐ *Conduct quality testing relevant to the project (security, performance, usability, accessibility, compatibility, reliability, or maintainability)*
- ☐ *Address at least 2 quality areas where applicable*
- ☐ *Use risk-based reasoning to justify the selected testing activities*
- ☐ *Document testing evidence, results, limitations, and recommended improvements*

7.1 Risk-Based Selection of Quality Areas

7.2 Quality Area One: [e.g. Security]

7.3 Quality Area Two: [e.g. Performance]

7.4 Results and Limitations

7.5 Recommended Improvements

8. Quality Metrics and Final Release Decision

- ☐ *Present relevant quality metrics (test pass rate, defect severity distribution, requirement coverage, test coverage, automation coverage, code quality indicators, commit activity)*
- ☐ *Evaluate the completed system against requirements, acceptance criteria, and quality attributes*
- ☐ *Summarise testing results, defect status, metrics, limitations, and residual risks*
- ☐ *State whether the system is ready for release, conditionally ready, or not ready for release*
- ☐ *Justify the release decision using evidence*

8.1 Final Quality Metrics

8.2 Requirements and Acceptance-Criteria Evaluation

8.3 Outstanding Defects and Residual Risks

8.4 Final Release Decision

9. AI-Assisted Development and Critical Reflection

- ☐ Explain how GitHub Copilot was used (implementation, testing, debugging, refactoring, or QA activities)
- ☐ Provide representative evidence of AI-assisted development
- ☐ Explain how AI-assisted outputs were reviewed, modified, tested, accepted, or rejected
- ☐ Identify quality risks found in AI-assisted outputs (incorrect logic, insecure code, poor maintainability, unsuitable assumptions)
- ☐ Reflect on how AI-assisted development affected productivity, learning, testing, software quality, and professional practice

9.1 AI-Assisted Activities

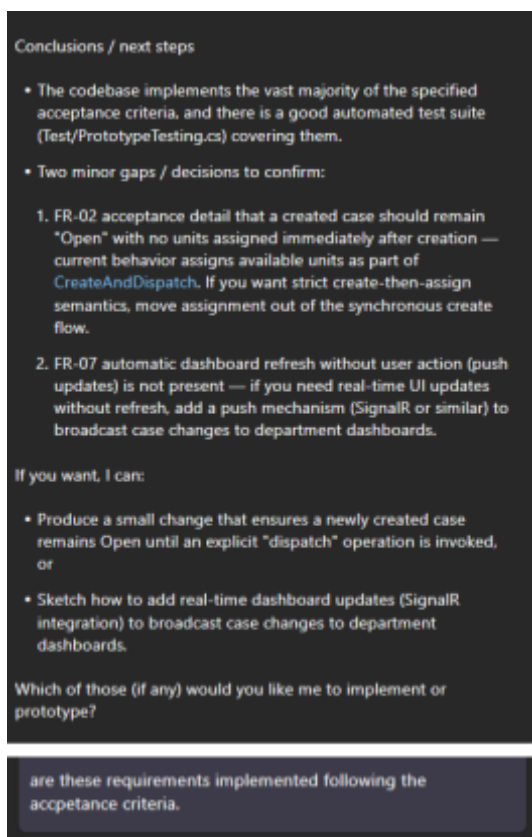


Figure 1.2.1: Github Copilot Evaluation

In the previous assignment, the functional requirements and non-functional requirements are defined and the acceptance criteria are created to describe expected behaviour of the core functional requirements. GitHub Copilot has been used to review the requirements that are gaps identified of core functionality with acceptance criteria. FR-02 and FR-07 are flagged as gaps that need to be considered for further changes. FR-01, FR-04, FR-05, FR-06, FR-08, FR-09 and FR-11 are not found as issues and are considered potential issues by GitHub Copilot. It identified that FR-02 acceptance criteria is not fully met, this criteria in FR-02 “Given a case is newly created from a recorded call, when creation completes, then the case status is set to “Open” with no units assigned to it yet”. FR-07 is flagged because GitHub described that the dashboard does not automatically refresh without user action, which doesn’t meet this: “Given a department’s dashboard is open, when the information of a case changes e.g. priority, location, then the dashboard should refresh and show the new changes automatically.” and “Given a department’s dashboard is open, when a case is routed to that department, then the dashboard should display the new case automatically.” When personally

investigating whether the FR-02 implementation meets the acceptance criteria gaps during testing, there isn't an issue because if there is a unit available, then the case would be assigned, thus receiving the status of "InProgress". The acceptance criteria is only met when there are no available units in that relevant department. Therefore, the acceptance criteria would need to be updated that the criteria is only for when there are no available units. For FR-07, after personal testing of the prototype, it has met its expected behaviour since the dashboard is updated without manually refreshing. The suggested GitHub flagged issues because it assumed that automatic updates mean adding a push mechanism (SignalR) to satisfy the stated criteria.

9.2 Representative GitHub Copilot Evidence

9.3 Human Review and Validation

9.4 AI-Related Quality Risks

9.5 Critical Reflection

10. Quality Improvement and Future Recommendations

- ☐ *Identify the main quality lessons learned from the project*
- ☐ *Recommend improvements to the development and QA process*
- ☐ *Propose future improvements to testing, automation, CI/CD, monitoring, security, usability, accessibility, or maintainability*
- ☐ *Explain how the project could evolve toward stronger continuous quality engineering practice*

10.1 Key Quality Lessons

10.2 Improvements to the Development and QA Process

10.3 Future Testing and Automation Improvements

10.4 Future Continuous Quality Engineering

11. Team Contribution and Professional Reflection

- ☐ *Provide a contribution statement for each group member*
- ☐ *Provide evidence from GitHub commits, project artefacts, testing work, documentation, and implementation tasks*
- ☐ *Include a brief individual reflection from each group member*

11.1 Contribution Statement

11.2 GitHub and Project Evidence

11.3 Individual Reflection – Member One

11.4 Individual Reflection – Member Two

11.5 Individual Reflection – Member Three

Delivarables (Notes)

Prepare a 15-slide presentation covering: title/members/problem/stakeholders/scope, key requirements, final prototype and changes since Assessment 1, testing approach/coverage/results, automation/CI/CD, defects and RCA, quality metrics and release decision, Copilot use and limitations, individual contributions and lessons learned

All students participate in the demonstration and show the working prototype

Demo includes key features, important test cases, automated tests/CI results, defect fixes, evidence major requirements are satisfied

Be prepared to answer questions on the project, code, testing, quality artefacts, AI-assisted development, and individual contributions

Deliverables

- Final Project and Software Quality Assurance Report: 6,000–7,000 words
- Source code and GitHub repository link, with evidence from GitHub commits
- Contribution statement for each group member
- Project presentation
- Turnitin Similarity Report

References:

O Aotearoa, R. I. T. R. I. (n.d.). Pressure grows to fix 111 emergency call system, with 1958 model still in use. RNZ.

https://www.rnz.co.nz/news/emergencies_local/585202/pressure-grows-to-fix-111-emergency-call-system-with-1958-model-still-in-use

1News Reporters. (2026, August 10). St John bracing for another 111 spike amid record demand. 1News.

<https://www.1news.co.nz/2026/08/10/st-john-bracing-for-another-111-spike-amid-record-demand/>

GeeksforGeeks. (2026, May 13). Repository design pattern. GeeksforGeeks.

<https://www.geeksforgeeks.org/system-design/repository-design-pattern/>

GeeksforGeeks. (2026, May 14). Strategy design pattern. GeeksforGeeks.

<https://www.geeksforgeeks.org/system-design/strategy-pattern-set-1/>